Project - High Level Design

On

Finance Support Triage Agent

Course Name: GenAI

Institution Name: Medi-caps University – DataGami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Aparna Sharma	EN24CA5030023
2.	Aditya Bairagi	EN24CA5030009
3.	Shubhanjali Roy	EN24CA5030161
4.	Mimansa Kashyap	EN22IT301054
5.	Harsh Gupta	EN24CA5030057
6.	Sandhya Patidar	EN24CA5030150

Group Name: Group – G-12D12

Project Number:GAI-22

Industry Mentor Name:

University Mentor Name: Ajit Singh Rajput

Academic Year:2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The **Finance Support Triage System** is an intelligent, AI-powered solution designed to automate and streamline the handling of customer queries in financial services such as banking, insurance, and fintech platforms. In today's fast-paced digital environment, organizations receive a large volume of support requests through multiple channels like email, chat, and APIs. Manually processing these requests is time-consuming, error-prone, and inefficient.

This project leverages advanced **Large Language Models (LLMs)** and **Natural Language Processing (NLP)** techniques to automatically classify, prioritize, and respond to customer queries. The system is capable of understanding user intent, extracting key entities (such as account details, transaction IDs, or issue types), and generating context-aware responses.

The core objective of this system is to improve operational efficiency by reducing response time, minimizing human intervention, and ensuring accurate query handling. It also enables intelligent routing of complex or high-priority issues to appropriate human agents through an automated workflow orchestration mechanism.

1.1 Scope of the document

This document provides a comprehensive overview of the **Finance Support Triage System**, including its design, architecture, functionalities, and implementation approach. It outlines how the system leverages **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)** to automate the

classification, prioritization, and handling of financial support queries.

The scope of this document covers the system's core components such as the preprocessing layer, classification engine, Named Entity Recognition (NER), context builder, response generator, and workflow orchestrator. It also describes the technologies used, data flow, and interaction between different modules.

Additionally, the document highlights the key features, expected outcomes, and potential use cases within financial domains such as banking, insurance, and fintech platforms. It is intended to guide developers, stakeholders, and users in understanding the system's capabilities and implementation details.

However, this document does not cover detailed deployment configurations, third-party integrations at production scale, or organization-specific compliance requirements, which may vary based on the implementation environment.

1.2 Intended Audience

This document is intended for a diverse group of stakeholders involved in the development, implementation, and usage of the **Finance Support Triage System**.

- **Developers and Engineers**: To understand the system architecture, components, and implementation details for building and maintaining the application.
- **Project Managers**: To gain insights into the project scope, workflow, and overall system capabilities for effective planning and coordination.
- **Business Analysts**: To evaluate how the system addresses financial support challenges and improves operational efficiency.

- **Financial Service Providers** (Banks, Insurance, Fintech Companies): To understand how the system can be integrated into existing customer support processes.
- **End Users / Support Teams**: To get an overview of how the system assists in automating query handling and improving response time.
- **Researchers and Students**: To study the application of AI, NLP, and LLMs in real-world financial systems.

1.3 System overview

The **Finance Support Triage System** is an AI-driven platform designed to automate the processing and management of customer support queries in financial domains such as banking, insurance, and fintech. The system utilizes **Large Language Models (LLMs)** and **Natural Language Processing (NLP)** techniques to understand, classify, and respond to user queries efficiently.

The system follows a modular architecture where incoming requests from multiple channels (such as email, chat, or APIs) are first passed through a **preprocessing layer** to clean and standardize the data. The processed input is then analyzed by a **classification engine**, which identifies the user's intent and determines the urgency level of the query.

Next, a **Named Entity Recognition (NER) module** extracts important information such as account details, transaction IDs, dates, or issue categories. This information is combined in the **context builder**, which prepares a structured representation of the query for further processing.

2. System Design

The **Finance Support Triage System** is designed using a modular and scalable architecture that integrates **AI/ML components** with traditional backend services. The system processes incoming financial queries through multiple stages, ensuring efficient classification, information extraction, and response generation..

The system follows a pipeline-based workflow:

1. **Input Layer**
   - Accepts queries from multiple sources such as email, chat, or API.
   - Supports unstructured and semi-structured data.
2. **Preprocessing Layer**
   - Cleans and normalizes text data.
   - Removes noise, stop words, and performs tokenization.
3. **Classification Engine (Intent + Urgency)**
   - Identifies the intent (e.g., complaint, inquiry, transaction issue).
   - Assigns urgency levels (low, medium, high).
4. **Named Entity Recognition (NER)**
   - Extracts key entities like:
     - Account Number
     - Transaction ID
     - Date & Amount
     - Issue Type

5. **Context Builder**
    - Combines extracted entities and intent.
    - Prepares structured input for the LLM.
6. **LLM Response Generator**
    - Generates intelligent, context-aware responses.
    - Uses prompt engineering and domain-specific tuning.
7. **Workflow Orchestrator**
    - Decides next action:
        - Auto-reply
        - Escalation to human agent
        - Ticket creation
8. **Output Layer**
    - Sends response back to user or support system.
    - Logs data for monitoring and improvement.

2.1 Application Design

The **Application Design** defines how the system is structured at the software level, including the interaction between different layers, modules, and technologies. The Finance Support Triage System follows a **multi-layered architecture** to ensure scalability, maintainability, and separation of concerns

The application is divided into the following layers:

A. Presentation Layer

- Built using **JSP, HTML, CSS, JavaScript**
- Provides user interface for:
  - Submitting queries (chat form / email input)
  - Viewing responses
- Communicates with the backend using **HTTP requests**

B. Controller Layer (Servlet Layer)

- Implemented using Java Servlets
- Acts as the bridge between UI and business logic
- Responsibilities:
  - Handle incoming requests
  - Validate input data
  - Route requests to appropriate services

C Service / Business Logic Layer

- Contains core logic of the application
- Modules:
  - Preprocessing Service
  - Classification Service
  - NER Service
  - Context Builder
  - LLM Integration Service
  - Workflow Manager
- Applies decision-making and orchestration logic

D Data Access Layer (DAO Layer)

- Uses DAO & DAOImpl pattern
- Interacts with database (e.g., MySQL)
- Performs:
  - Insert (queries, logs, tickets)
  - Fetch (history, responses)
  - Update (status, escalation)

E Database Layer

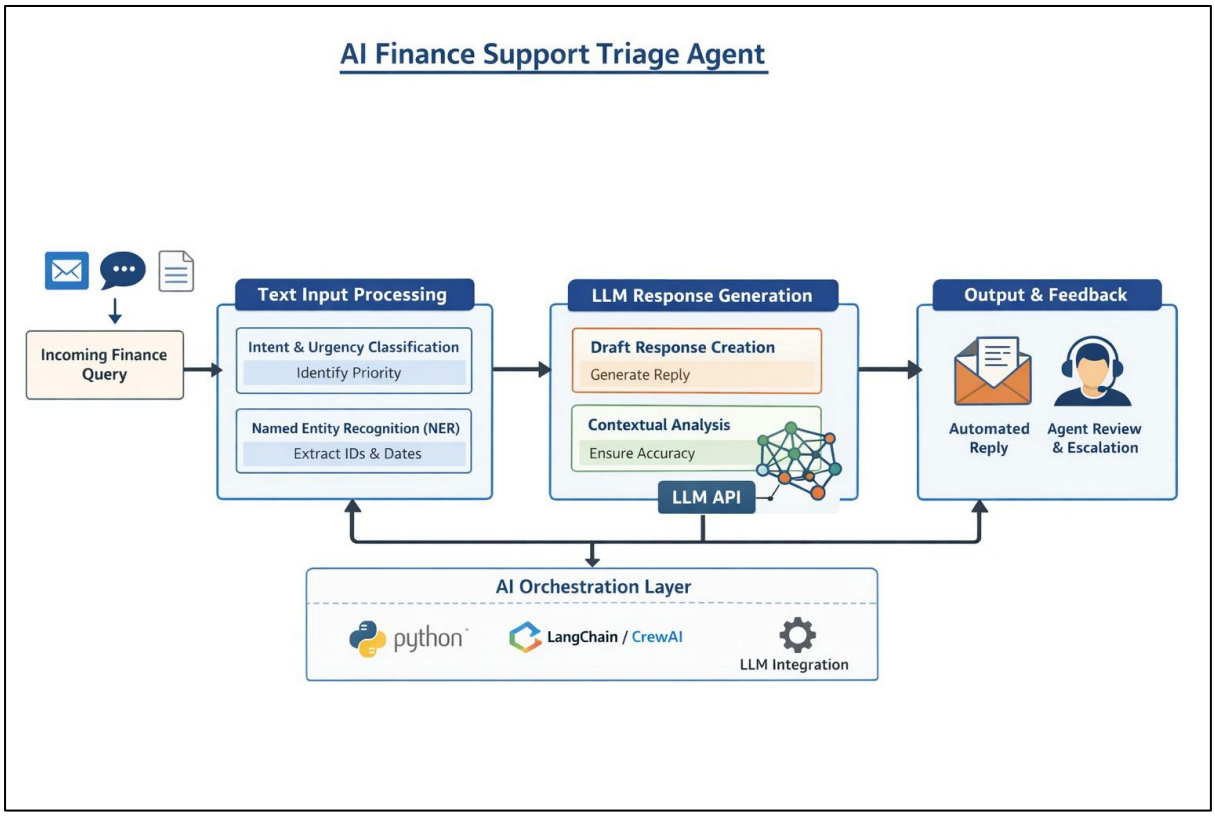
- Stores:
  - User data
  - Query logs
  - Tickets
  - Responses
- Example: MySQL Database (aditya → users table, logs, etc.)

2.2 Process Flow

The **Finance Support Triage System** follows a structured and automated process to handle customer queries efficiently. The process begins when a user submits a query through a chat interface, email, or API. This input is first passed through a preprocessing stage where the text is cleaned, normalized, and prepared for analysis. The system then uses an intelligent classification engine to identify the intent of the query (such as complaint, request, or inquiry) and determine its urgency level. After that, a Named Entity Recognition (NER) module extracts important details like transaction ID, account number, amount,

and date. These elements are combined in the context-building phase to create a structured representation of the query.

Once the context is prepared, it is sent to the LLM-based response generator, which produces a relevant and context-aware reply. Based on the complexity and priority of the query, the workflow orchestrator decides whether to send an automated response, escalate the issue to a human agent, or generate a support ticket. Finally, the system logs all relevant data in the database and delivers the response back to the user. This end-to-end process ensures fast, accurate, and intelligent handling of financial support queries while minimizing manual intervention.



2.3 Information Flow

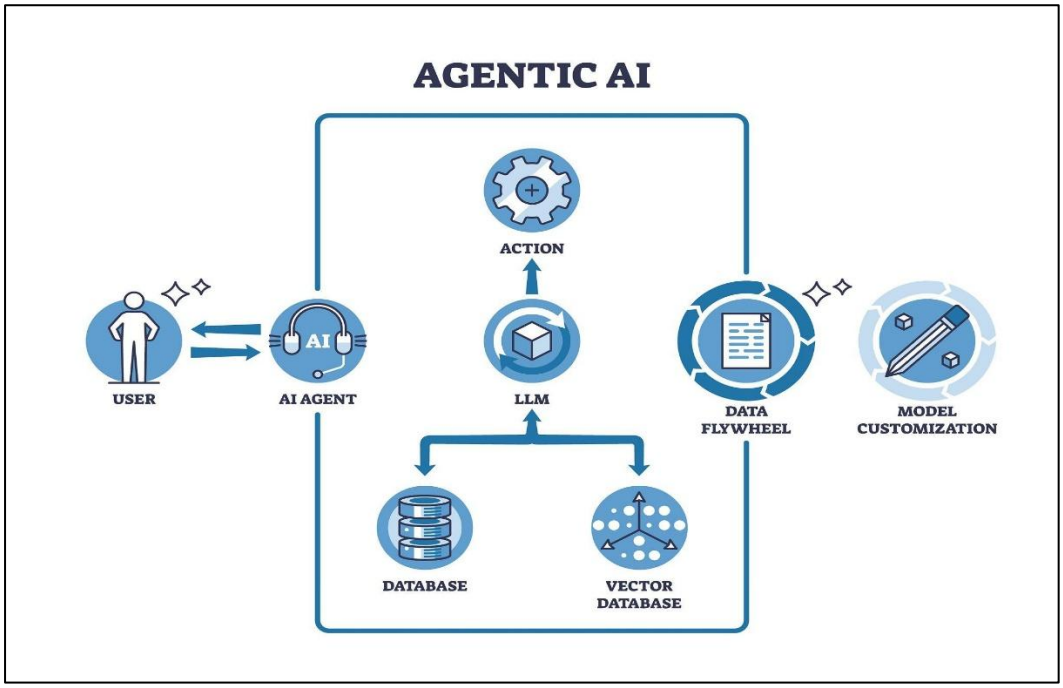
The **Information Flow** in the Finance Support Triage System describes how data moves and transforms across different components of the application. The process begins when a user submits a query, which is received by the system and passed to the preprocessing module for cleaning and normalization. The refined data is then forwarded to the classification engine, where the intent and urgency of the query are identified. This information is further enriched by the Named Entity Recognition (NER) module, which extracts key entities such as transaction details, account numbers, and dates.

The processed data, including intent, urgency, and extracted entities, is then combined in the context builder to form a structured input. This structured information flows into the LLM-based response generator, which produces an appropriate and context-aware reply. Simultaneously, the workflow orchestrator uses this information to determine the next course of action, such as auto-response, escalation, or ticket creation. All relevant data is then stored in the database for tracking, auditing, and future analysis. Finally, the generated response is sent back to the user through the respective communication channel. This seamless flow of information ensures efficient data handling, accurate decision-making, and improved system performance.

2.4 Component Design

The **Component Design** of the Finance Support Triage System defines the internal modules of the application and how each component performs a specific function within the overall workflow. The system is divided into loosely coupled components, each responsible for handling a distinct part of

the query processing pipeline, ensuring scalability, maintainability, and efficiency.



Core Components Description

1 **Input Handler Component**

- Receives user queries from multiple sources (chat, email, API)
- Validates and forwards input to preprocessing module

2 **Preprocessing Component**

- Cleans and normalizes raw text
- Removes noise, performs tokenization
- Prepares data for analysis

3 **Classification Component**

- Uses AI/ML models to determine:
    - Intent (complaint, query, request)

    - Urgency (low, medium, high)

4 **NER (Named Entity Recognition) Component**

- Extracts key entities such as:
    - Account Number
    - Transaction ID
    - Amount and Date
- Converts unstructured data into structured form

5 **Context Builder Component**

- Combines:
    - User input
    - Extracted entities
    - Intent and urgency
- Creates a structured context for further processing

6 **LLM Integration Component**

- Connects with external AI models (LLM APIs)
- Generates intelligent, human-like responses
- Uses prompt engineering for better accuracy

7 **Workflow Orchestrator Component**

- Controls system decision-making
- Determines:
    - Auto-response
    - Escalation to human agent
    - Ticket generation

8 **Database Component**

- Stores:

  - User queries
  - Responses
  - Logs and tickets
- Ensures data persistence and retrieval

9 **Output Handler Component**

- Sends response back to user
- Updates dashboards or support systems

2.5 Key Design Considerations

The design of the **Finance Support Triage System** is based on several critical factors to ensure that the application performs efficiently and meets real-world financial requirements. One of the primary considerations is **security and data privacy**, as the system handles sensitive financial information such as account details and transaction data. Proper encryption, authentication, and secure communication mechanisms are essential to protect user data.

Another important factor is **performance and scalability**. The system is designed to handle a large number of user queries simultaneously without affecting response time. A modular and layered architecture allows the system to scale easily as demand increases. Additionally, **accuracy and reliability** of AI components, such as intent classification, Named Entity Recognition (NER), and response generation, are crucial for delivering correct and consistent results.

The system also emphasizes **modularity and maintainability**, where each component operates independently, making it easier to update or modify specific modules without impacting the entire

system. **Integration capability** is another key consideration, ensuring that the application can seamlessly connect with external systems like banking services, CRM platforms, and ticketing tools.

2.6 API Catalogue

The **API Catalogue** defines the set of application programming interfaces used in the **Finance Support Triage System** to enable communication between different components such as the frontend, backend, AI modules, and external services. These APIs follow RESTful principles and use standard HTTP methods for data exchange.

List of APIs

1 Submit Query API

- **Endpoint**: /api/query/submit
- **Method**: POST
- **Description**: Accepts user queries from the UI or external systems

2. Preprocessing API

- **Endpoint**: /api/query/preprocess
- **Method**: POST
- **Description**: Cleans and normalizes the input text
- **Input**: Raw query text
- **Output**: Processed text

3. Classification API

- **Endpoint**: /api/query/classify
- **Method**: POST
- **Description**: Identifies intent and urgency of the query

4. NER Extraction API

- **Endpoint**: /api/query/ner
- **Method**: POST
- **Description**: Extracts key entities from the query

5. Context Builder API

- **Endpoint**: /api/query/context
- **Method**: POST
- **Description**: Combines intent, urgency, and entities into structured context

6. LLM Response API

- **Endpoint**: /api/query/respond
- **Method**: POST
- **Description**: Generates AI-based response using LLM

7. Workflow Decision API

- **Endpoint**: /api/query/decision
- **Method**: POST
- **Description**: Determines whether to auto-reply, escalate, or create ticket

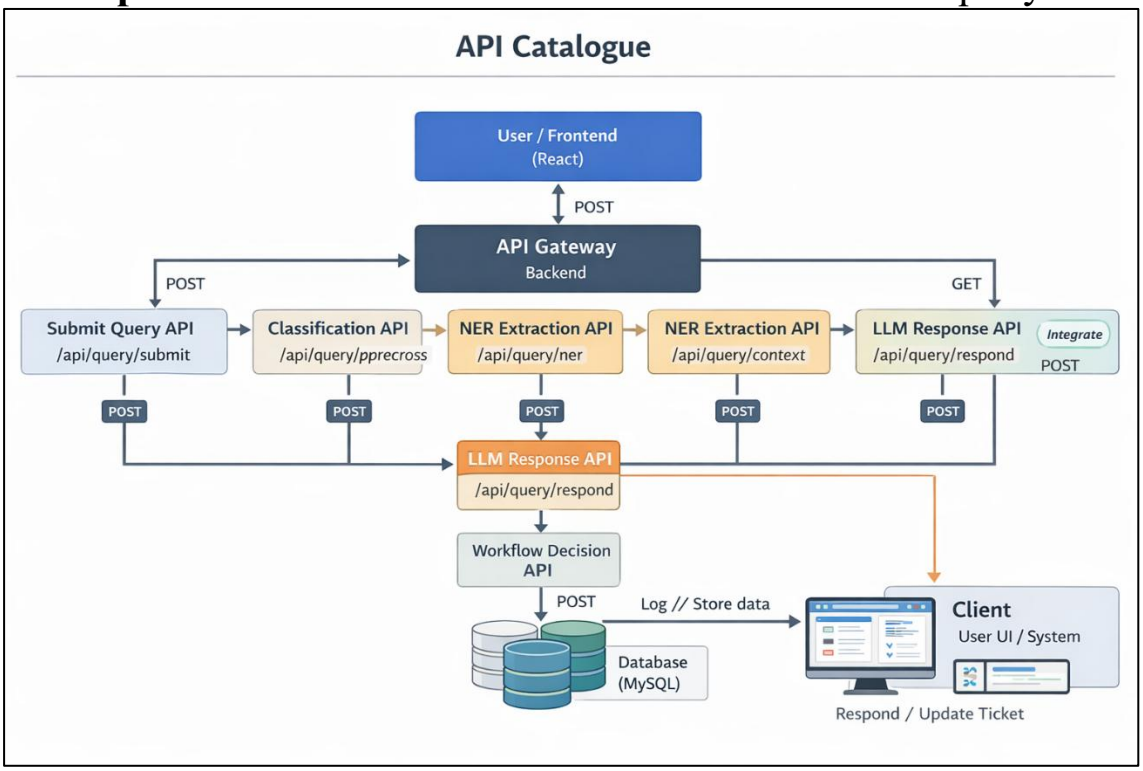
8. Ticket Creation API

- **Endpoint**: /api/ticket/create
- **Method**: POST
- **Description**: Creates a support ticket for unresolved or high-priority queries

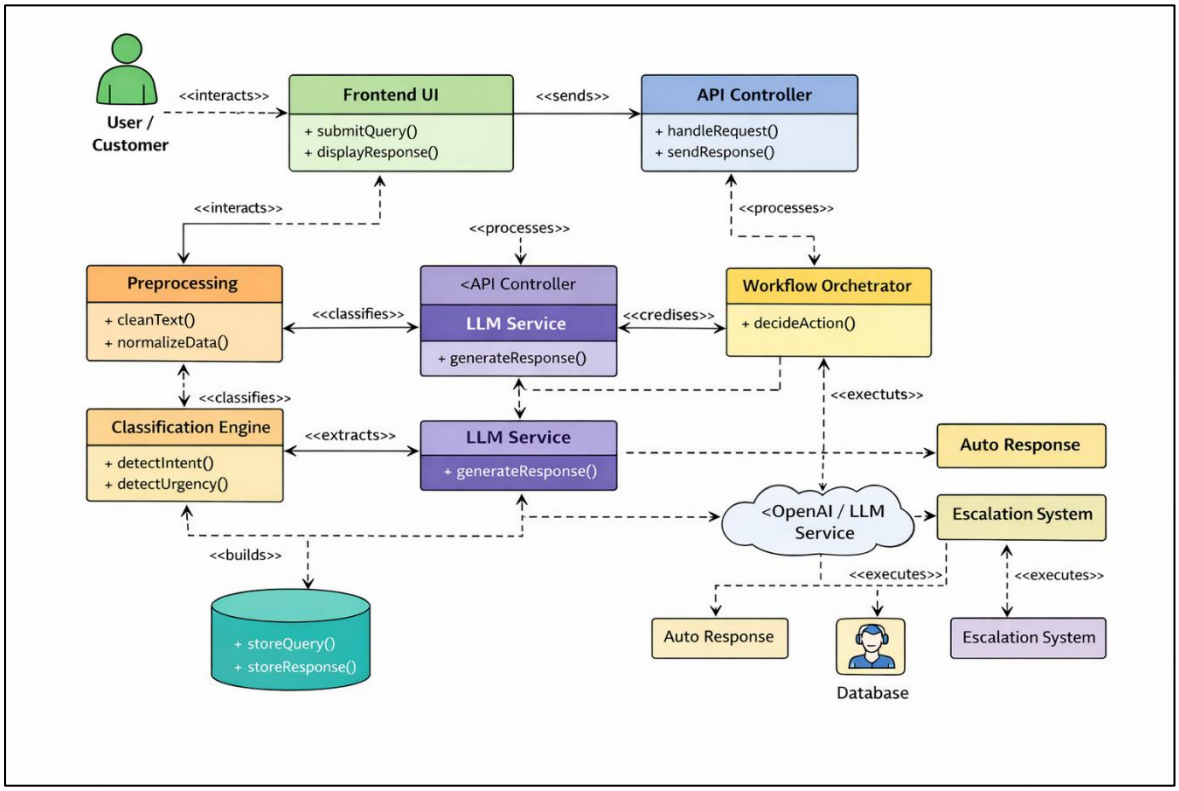
9. Query Status API

- **Endpoint**: /api/query/status/{queryId}
- **Method**: GET

- **Description**: Retrieves the current status of a query



UML Diagram



3.Data Design

The **Data Design** of the Finance Support Triage System defines how data is structured, stored, and managed to support efficient processing of financial support queries. The system uses a **relational database (MySQL)** to maintain consistency, integrity, and scalability of data across different modules.

The design focuses on storing user information, query details, extracted entities, generated responses, and ticket data in a structured manner. Each type of data is organized into separate tables, with relationships established using primary and foreign keys to ensure proper linkage between records.

The **Users table** stores basic information about customers, such as name, contact details, and login credentials. The **Queries table** maintains all user-submitted queries along with attributes like message content, detected intent, urgency level, and status. To support intelligent processing, the **Entities table** stores key information extracted through Named Entity Recognition (NER), such as transaction ID, account number, amount, and date.

The system also includes a **Responses table**, which stores AI-generated replies for each query, enabling tracking and analysis of system outputs. In cases where queries require human intervention, a **Tickets table** is used to manage escalated issues, including details such as assigned agent, priority, and resolution status.

Relationships between these tables are carefully defined—for example, a single user can have multiple queries, each query can have associated entities and responses, and may also generate a ticket if escalated. Constraints such as primary keys, foreign keys, and indexing are applied to ensure data integrity and improve performance.

3.1 Data Model

The **Data Model** of the Finance Support Triage System defines the logical structure of data and the relationships between different entities in the system. It represents how data is organized within the database to support efficient query processing, storage, and retrieval.

The system follows a **relational data model**, where data is stored in structured tables with clearly defined relationships using primary and foreign keys. The main entities in the system include **Users, Queries, Entities (NER Output), Responses, and Tickets**.

Each **User** can submit multiple queries, establishing a one-to-many relationship between the Users and Queries tables. The **Queries** entity acts as the central component, storing the user's message along with attributes such as detected intent, urgency level, and status. For every query, the system may extract multiple pieces of information (like transaction ID, amount, and date), which are stored in the **Entities** table, creating a one-to-many relationship.

The **Responses** table maintains the output generated by the LLM for each query, typically forming a one-to-one relationship with the Queries table. In cases where the system identifies high-priority or complex issues, a **Ticket** is created and stored in the Tickets table, linking back to the corresponding query.

The data model ensures normalization to reduce redundancy and maintain consistency. Relationships between entities are enforced using foreign key constraints, while indexing is applied to improve query performance. This structured approach enables efficient data handling and supports the overall functionality of the system.

3.2 Data Access Mechanism

The **Data Access Mechanism** of the Finance Support Triage System defines how the application interacts with the database to perform operations such as storing, retrieving, updating, and managing data. The system follows a structured approach using the **DAO (Data Access Object) design pattern**, which separates business logic from database operations and ensures better maintainability and scalability.

The application uses **Java (JSP/Servlet)** for backend processing and **MySQL** as the database. The DAO layer acts as an intermediary between the service layer and the database, handling all CRUD (Create, Read, Update, Delete) operations. Each major entity in the system—such as Users, Queries, Responses, Entities, and Tickets—has a corresponding DAO and DAO Implementation (DAOImpl) class.

The system uses **JDBC (Java Database Connectivity)** to establish a connection with the database. A dedicated utility class (e.g., DBConnection.java) is responsible for managing database connections efficiently. SQL queries are executed through prepared statements to ensure security and prevent SQL injection attacks.

For example, when a user submits a query, the Servlet passes the request to the service layer, which then calls the **QueryDAO** to store the data in the database. Similarly, when a response is generated, the **ResponseDAO** is used to save and retrieve response data. In case of escalation, the **TicketDAO** manages ticket creation and updates.

The mechanism also includes proper **error handling and transaction management** to maintain data consistency. Logging mechanisms are used to track database operations for monitoring and debugging purposes.

3.3 Data Retention Policies

The **Data Retention Policies** of the Finance Support Triage System define how long different types of data are stored, how they are managed over time, and when they are archived or deleted. These policies ensure efficient storage utilization, regulatory compliance, and protection of sensitive financial information.

The system categorizes data into different types such as user data, query logs, extracted entities, responses, and support tickets. Each category follows a defined retention period based on its importance and usage.

User account information is retained for as long as the account is active, while inactive accounts may be archived or removed after a specified period of inactivity. Query logs and responses are typically stored for a limited duration (e.g., 6–12 months) to support auditing, analytics, and performance improvement of AI models. Extracted entity data, which may contain sensitive financial details, is stored securely and retained only as long as necessary to process and resolve the query.

Support tickets and escalation records are maintained for a longer duration to ensure traceability, compliance, and customer service history tracking. After the retention period expires, data may be archived for backup purposes or permanently deleted following secure data disposal practices.

The system also incorporates **data anonymization and encryption techniques** to protect user privacy. Access to stored data is controlled through authentication and authorization mechanisms to prevent unauthorized usage.

3.4 Data Migration

The **Data Migration** process in the Finance Support Triage System defines how data is transferred from existing systems or external sources into the new system without loss, inconsistency, or corruption. This is especially important when integrating with legacy banking systems, CRM platforms, or previous support databases.

The migration process begins with **data extraction**, where relevant data such as user details, historical queries, transaction records, and support tickets are collected from the source systems. This data is then passed through a **data transformation** phase, where it is cleaned, formatted, and mapped to match the schema of the new system. During this stage, inconsistencies, duplicates, and invalid entries are identified and resolved.

After transformation, the data is loaded into the target database using structured scripts or automated tools, following the **ETL (Extract, Transform, Load)** approach. Proper validation checks are performed to ensure that the migrated data maintains integrity, accuracy, and completeness. Foreign key relationships and constraints are also verified to maintain consistency across tables.

The system includes **backup and rollback mechanisms** to prevent data loss in case of migration failures. Additionally, migration is often performed in phases (e.g., test migration, pilot run, and final migration) to minimize risks and ensure smooth transition.

Security is a key consideration during migration, with sensitive financial data being encrypted and transferred through secure

channels. Access controls are enforced to ensure that only authorized personnel can perform migration activities.

4. Interfaces

The **Interfaces** of the Finance Support Triage System define how different components of the system interact with users, internal modules, and external services. The system includes a user-friendly **User Interface (UI)** developed using JSP, HTML, CSS, and JavaScript, which allows users to submit queries and receive responses in an interactive manner. The **API Interface** enables communication between the frontend and backend through RESTful services, handling operations such as query submission, processing, and response delivery. Additionally, the system integrates with an **AI/LLM Interface**, which connects to external language models to generate intelligent and context-aware responses based on user input. The **Database Interface**, implemented using JDBC and the DAO pattern, ensures efficient and secure interaction with the MySQL database for storing and retrieving data such as queries, responses, and tickets. Furthermore, the system supports **External System Interfaces** for integration with banking systems, CRM platforms, and ticketing tools, enabling seamless data exchange and workflow automation. Together, these interfaces ensure smooth communication, efficient processing, and a reliable user experience across the entire system.

5. State and Session Management

The **State and Session Management** in the Finance Support Triage System ensures that user interactions are maintained consistently across multiple requests and that relevant data is preserved throughout a user's session. Since web applications are inherently stateless, the system uses session management

techniques to track user activities and maintain context during interactions.

The system utilizes **HTTP Sessions** in Java Servlets to store temporary user-specific data such as user ID, login status, and active query information. When a user logs in or submits a query, a unique session is created and maintained on the server, allowing the system to identify and manage subsequent requests from the same user. This helps in providing a personalized and continuous experience.

For managing conversational context, especially in AI-based interactions, the system maintains session-level data such as previous queries, extracted entities, and generated responses. This enables the LLM to generate more accurate and context-aware replies. Session timeouts are configured to automatically invalidate inactive sessions after a specific period, improving security and resource management.

Additionally, the system may use **cookies** or token-based mechanisms (such as JWT) for authentication and session tracking in distributed environments. Sensitive session data is secured using encryption and proper access controls to prevent unauthorized access.

6. Caching

The **Caching** mechanism in the Finance Support Triage System is used to improve system performance, reduce response time, and minimize repeated processing of similar requests. Since the system involves AI-based processing and database interactions, caching plays a crucial role in optimizing resource utilization and ensuring faster responses.

The system implements caching at multiple levels. At the **application level**, frequently accessed data such as user session details, recent queries, and common responses are stored temporarily in memory. This avoids repeated database queries and reduces latency. For example, if a user submits a similar query within a short time, the system can return a cached response instead of reprocessing it through the entire pipeline.

At the **API level**, responses from external services such as the LLM can be cached for commonly asked questions. This reduces the number of API calls, lowers operational cost, and improves response speed. Cache entries are managed using strategies like **Time-To-Live (TTL)** to ensure that outdated data is automatically refreshed.

The system may also use **database query caching** to store results of frequently executed queries, improving data retrieval performance. In distributed environments, tools like **Redis or in-memory caching systems** can be used to provide fast and scalable caching support.

Proper **cache invalidation policies** are implemented to maintain data consistency. For instance, when a query status is updated or a ticket is resolved, related cache entries are cleared or updated. Additionally, sensitive financial data is either not cached or stored securely to maintain privacy and compliance.

7. Non-Functional Requirements

The **Non-Functional Requirements** of the Finance Support Triage System define the quality attributes and performance standards that the system must meet to ensure reliability, efficiency, and user satisfaction. These requirements focus on how the system operates rather than what it does.

The system must provide high **performance** by processing user queries quickly and generating responses with minimal latency, even under heavy load. It should be **scalable**, allowing it to handle an increasing number of users and requests without degradation in performance. The system must also ensure high **availability**, remaining operational and accessible with minimal downtime.

Security is a critical requirement, as the system handles sensitive financial data. It must implement strong authentication and authorization mechanisms, secure data transmission using HTTPS, and protect stored data through encryption. Additionally, proper access control must be enforced to prevent unauthorized usage.

The system should maintain high **reliability**, ensuring consistent and accurate results from AI components such as classification, NER, and response generation. In case of failures or uncertainties, fallback mechanisms like escalation to human agents must be available. **Usability** is also important, requiring a simple and intuitive user interface that enhances user experience.

7.1 Security Aspects

The **Security Aspects** of the Finance Support Triage System are critical due to the handling of sensitive financial and personal data. The system is designed with multiple layers of security to ensure data confidentiality, integrity, and availability.

The application implements strong **authentication and authorization** mechanisms to ensure that only authorized users can access system resources. User credentials are securely stored using encryption or hashing techniques, and role-based access control (RBAC) is used to restrict access based on user roles such as admin, support agent, or customer.

All communication between the client, server, and external

services is secured using **HTTPS (SSL/TLS encryption)** to prevent data interception. Sensitive data such as account numbers and transaction details are either encrypted or masked before storage and transmission.

To protect against common web vulnerabilities, the system includes safeguards such as **input validation and sanitization** to prevent SQL injection, cross-site scripting (XSS), and other malicious attacks. Prepared statements are used in database operations to enhance security.

The system also enforces **session management security**, including session timeouts and secure cookies, to prevent session hijacking. In distributed environments, token-based authentication methods such as JWT can be used for secure access control.

Additionally, **logging and monitoring mechanisms** are implemented to track suspicious activities and system access. Regular audits and security checks help identify and mitigate potential threats.

7.2 Performance Aspects

The **Performance Aspects** of the Finance Support Triage System focus on ensuring fast response times, efficient resource utilization, and the ability to handle a high volume of concurrent user requests. Since the system processes real-time financial queries and integrates AI-based components, maintaining optimal performance is essential for a smooth user experience.

The system is designed to deliver **low latency responses**, where user queries are processed and answered within a short time frame. This is achieved through efficient preprocessing, optimized API calls, and the use of caching mechanisms to avoid repeated computations. Frequently accessed data and common query responses are cached to reduce processing overhead and improve speed.

To support **scalability**, the system follows a modular and distributed architecture, allowing it to handle increasing

workloads by adding more resources. Load balancing techniques can be used to distribute incoming requests across multiple servers, ensuring consistent performance even during peak usage.
The application also ensures **efficient database performance** by using indexing, optimized queries, and connection pooling to reduce query execution time. Background processing and asynchronous operations are utilized for tasks such as logging and ticket creation, preventing delays in user response.
Performance monitoring tools are integrated to track system metrics such as response time, throughput, and resource utilization. This helps in identifying bottlenecks and optimizing system behavior over time.

References

The following references were used as guidance for designing and developing the **Finance Support Triage System**:

1. **Ian Goodfellow, Yoshua Bengio, Aaron Courville** – *Deep Learning*, MIT Press, 2016.
    - For understanding machine learning and neural network concepts.
2. **Daniel Jurafsky & James H. Martin** – *Speech and Language Processing*, Pearson.
    - For Natural Language Processing (NLP) techniques such as text processing and NER.
3. **Tom Mitchell** – *Machine Learning*, McGraw-Hill.
    - For foundational concepts in supervised and unsupervised learning.
4. Official Documentation – *OpenAI API Documentation*

- For integrating Large Language Models (LLMs) into the system.

5. **Oracle** – *Java Servlet and JSP Documentation*
   - For backend development using Java, Servlets, and JSP.
6. **MySQL Documentation** – *MySQL Reference Manual*
   - For database design, queries, and optimization.
7. **Martin Fowler** – *Patterns of Enterprise Application Architecture*
   - For understanding design patterns like DAO, MVC, and layered architecture.
8. **Roy Fielding** – *REST Architectural Style (REST APIs)*
   - For designing RESTful APIs used in the system.
9. Research Papers and Articles on:
   - AI-based customer support systems
   - Chatbots and conversational AI
   - Agentic AI and workflow automation

10 Online Resources:

- Tutorials from platforms like GeeksforGeeks, W3Schools, and official developer blogs
- Documentation and examples related to NLP, LLMs, and API integration